LAlgo: what we need to do

A quick guide for the team | 5 November 2026

1. What matters now

The lexical analyzer is working locally. Our main work is to turn the proposed language forms into a **formal grammar and a working syntax analyzer**. This guide condenses the four recordings and both checking rubrics. Short, timestamped excerpts support the requirements; full transcripts stay in the meeting notes.

READY means the stated portion exists in the current source or documentation. **It is not an instructor grade.** The lexer passes **31/31 local tests**.

FOCUS means missing work, partial coverage, or a decision we still need to settle. A keyword or sample program does not establish a working parser.

Settle these scope decisions first

1. **Choose one loop kind: `WHILE` or `FOR`.** Support both **single and nested** uses of that chosen kind. The current design lists both kinds.
2. **Keep language features through iteration only.** Review future-reserved `ALGORITHM`, `RETURN`, `TRACE`, and `INVARIANT`; do not expand them into extra supported constructs for this scope.
3. **Keep declarations in the work plan.** They are already in LAlgo's proposal. Define their grammar and parsing behavior.

Ma'am: "Kung FOR or WHILE, bahala kayo. Pero ang rule niya, dalawa."
Recording 04, 05:24–05:46. She then names single and nested loops.

Dates stated in the meeting

Date, 2026	Milestone
12 November	Submit syntax proposal: grammar, examples, and derivations. Face-to-face by default.
19 November	Asynchronous project work.
26 November	Lexical checking 2 and syntax checking 1.
3 December	Asynchronous project work.
10 December	Syntax checking 2, semantics checking 1, final project checking and final exam.

Team priority: scope decisions → grammar and per-rule derivations → parser and diagnostics. The iteration limit concerns language features; syntax and semantic checking milestones still apply. Submission time/platform and the full semantic rubric are not supplied in these materials.

2. Lexical analyzer: what already works

READY Recognition and output are implemented. The rows below map to the 11 lexical-rubric areas. They describe lexical behavior, not syntax or execution.

#	Area	Existing evidence — READY
1	Input restriction	Accepts `.lal` / `.LAL`; rejects other extensions before opening the source. Rejects NUL-containing input.
2	Identifiers	Letter first; then letters, digits, or underscores. Case-sensitive.
3	Keywords	Recognizes the current type, control-flow, and input/output vocabulary through the unified DFA.
4	Reserved words	Reserved spellings and separate future-reserved token classes exist. Their final scope needs review below.
5	Constant values	Five categories: integer, decimal, character, string, Boolean. `TRUE`/`FALSE` count as one category.
6	Noise words	Recognizes `SET` and `THE`.
7	Comments	Recognizes `//` line comments and `/* ... */` block comments; retains them in the listing.
8a	Arithmetic: 7	`+ - * / % DIV ^`. Assignment `=` is counted separately.
8b	Boolean: 9	Logical: `AND OR NOT`. Relational: `== != < > <= >=`.
9	Delimiters/brackets	Recognizes `( ) [ ] { }`, comma, semicolon, colon, and dot. Pairing/nesting checks belong to parsing.
10	Invalid tokens	Emits `INVALID` and lexical line/column diagnostics for documented malformed lexemes.
11	Output text file	Writes `Symbol Table.txt`: token occurrences, classifications, positions, and lexemes; includes comments/invalid entries.

Ma'am: "Sa lexical analyzer n'yo, na ang output ay symbol table."
Recording 04, 00:50–00:57. Retain the lexical output file.

Focus before presenting this as complete

- **Align the vocabulary.** Resolve the loop choice and the future-reserved words against the iteration scope boundary. Explain keyword versus reserved-word terminology.
- **Resolve rubric interpretation.** The extension filter works, but global uniqueness of `.lal` is unverified. Confirm the intended keyword/reserved distinction and nine-Boolean-operator classification.
- **Reconcile previous feedback.** Local tests do not establish that every item from Ma'am's previous checking has been addressed.

Demo evidence to keep: a valid `.lal` sample, a rejected `.txt`, all token categories, malformed input, and the generated `Symbol Table.txt`. The blank rubric scores remain Ma'am's assessment.

3. Syntax: what we still need

FOCUS All statement families still need formal grammar and parser support. Existing snippets provide examples, but no implemented syntax analyzer is present.

Family	Coverage Ma'am expects	Our next action
Input /1	One input construct; its rule design is up to us.	**FOCUS** Define `INPUT`'s grammar.
Output /3	String only; variable value only; string plus variable value.	**FOCUS** Define combined output; cover all three forms.
Assignment /3	Variable to variable; constant to variable; expression result to variable.	**FOCUS** Cover all three plain-assignment forms.
Condition /3	Single IF; IF–ELSE; IF–ELSE IF–ELSE chain.	**FOCUS** Define all three IF forms.
Iteration /2	Single loop and nested loops, using one chosen loop kind.	**FOCUS** Choose FOR or WHILE; cover nesting.
Declaration	Required for us because it is already proposed. Discuss single-variable, list, and initialized forms.	**FOCUS** Define declarations and list syntax.

Ma'am: "One example for every rule in that statement."
Recording 02, 01:02–01:08. Each rule needs its own matching example.

What each applicable rule needs for 12 November

1. **Formal grammar components:** terminals, nonterminals, productions, and a grammar start symbol. The lexer's `q0` is not that start symbol.
2. **A distinct matching example:** three examples of a single IF do not cover three different conditional forms. Cover additional applicable rules too.
3. **Leftmost and rightmost derivations:** apply the actual written productions until the example is obtained. Include the requested parse-tree/arrow-based derivation display.

READY Documented groundwork only: token vocabulary, expression examples, operator meanings, and intended precedence/associativity already exist in the specification. Reuse these decisions, then encode them in the grammar. They are not implemented evaluation.

Keep these distinctions clear. `a = b + 3` is plain assignment; `INTEGER a = b + 3` is declaration with initialization. The declaration count is not fixed on the screen; Ma'am tentatively discusses three forms. Her printf examples do not force C-style formatting in LAlgo.

4. Build the parser

READY Available foundation: the lexer stores token type, lexeme, line, and column in memory and creates EOF. Ma'am permits an **in-memory handoff**, one grammar terminal at a time. Continue generating `Symbol Table.txt` for the lexical rubric.

Required work — all marked Focus

1. Define the terminal interface.
Expose sequential token consumption, including EOF and source positions. Decide how comments, invalid entries, and noise words are handled. Define statement boundaries.

2. Select and implement a parser algorithm.
Recursive descent is an accepted example, not the only permitted algorithm. Document the chosen approach and make it enforce our grammar, precedence, and nesting.

3. Show derivations for valid syntax.
Provide the requested production/derivation output. A lexical DFA trace does not satisfy this requirement. Check every required statement form on page 3.

4. Name and implement error recovery.
Ma'am illustrates panic recovery. Choose grammar-appropriate synchronization points and show that checking continues after an invalid statement. Retain the overall error result.

5. Make syntax errors brief and useful.
Include location and the reason or expected construct. Her example is "Syntax error at line 15, semicolon expected." It does not itself mandate semicolons in our grammar.

Ma'am: "The error message must be brief and concise, but it will further give the programmer the reason why there is such an error."
Recording 04, 07:38–07:48. Include a useful reason, not just "syntax error."

Optional artifacts — not automatic blockers

Syntax transition diagram and parsing table are optional in the checking guide. Their usefulness depends on the approach we choose. Existing lexical DFA diagrams and the `DELTA` transition table are different artifacts.

Recommended team handoff order

1. **Agree:** one loop kind, supported statement forms, output/list syntax, boundaries, and reserved vocabulary. Compare relevant C/Java operations; Python is optional.
2. **Document:** Part 3 grammar plus a rule-to-example-and-derivation checklist. Preserve the existing worked lexical example when reorganizing the specification.
3. **Implement and check:** parser interface, required syntax, derivation output, error recovery, and concise diagnostics. Include single/nested-loop cases and malformed input.

Where this comes from. The 5 November recordings 01–04, both checking guides, and Adriel's reported iteration scope limit. Current source baseline: `e042368`; lexer tests: 31/31 locally. Quoted excerpts are lightly edited machine-assisted transcriptions; recording numbers follow the supplied files 1–4, with timestamps restarting per file. The iteration scope limit was reported by Adriel without a verified audio timestamp. Full transcripts and audit notes remain in the unified meeting notes. This is a planning guide, not an instructor-approved grade or completed grammar/parser.